Ingeniería en Tecnologías de la Información y las Comunicaciones

Tecnologías Inalámbricas

12:00 p m

3 Práctica NFR24L01 - Envío de datos con generador de señal

Profesor: Ricardo Alejandro Rodríguez Jiménez

Equipo:

Ramírez Rodríguez Martín de Jesús 23151193

Santacruz Rodarte Cristian Alexis 22150108

Pedroza Pérez Emmanuel 23151202

Fecha: 18 de marzo de 2026

Índice

Introducción 3

Descripción de la Esp32 4

Descripción de la NRF24L01 4

Diferenciación entre las NRF24L01 con y sin PA+LNA con y sin antena 5

Circuitos.....6

1. Transmisor 6

a) Diseño 6

b) Código 7

2. Receptor.....10

c) Diseño 10

d) Código 11

Conclusiones 15

Enlace a GitHub.....15

Introducción

Este trabajo comienza la integración de sensores en sistemas embebidos que permitirán ampliar las posibilidades de interacción y control en proyectos de comunicación inalámbrica. En este caso, se emplea un módulo ultrasónico HC-SR04 para detectar proximidad y transmitir la información a través de un ESP32 hacia otro dispositivo remoto, donde se activa un LED como respuesta. Este ejemplo reflejará cómo la combinación de microcontroladores, módulos de radiofrecuencia y sensores abre el camino hacia aplicaciones más complejas en el ámbito del Internet de las Cosas (IoT).

Descripción de la Esp32

La ESP32 es un microcontrolador versátil con Wi-Fi y Bluetooth integrados:

Características principales:

Procesador dual-core Tensilica LX6 a 240 MHz.

Conectividad Wi-Fi (802.11 b/g/n) y Bluetooth/BLE integrada.

Amplia cantidad de GPIOs para sensores, actuadores y periféricos.

Soporte para protocolos como SPI, I2C, UART, PWM y ADC.

Ventajas: Ideal para proyectos IoT, bajo costo, gran comunidad de soporte y capacidad de ejecutar tareas complejas con conectividad inalámbrica.

Descripción de la NRF24L01

NRF24L01 es un transceptor de 2.4 GHz diseñado para comunicaciones de bajo costo y bajo consumo.

Tipo: Transceptor inalámbrico de 2.4 GHz.

Funciones:

Comunicación bidireccional (envío y recepción).

Velocidades configurables: 250 kbps, 1 Mbps y 2 Mbps.

Bajo consumo de energía, adecuado para dispositivos portátiles.

Uso típico: Comunicación entre microcontroladores (Arduino, ESP32, STM32) en proyectos de domótica, sensores inalámbricos y control remoto.

Diferenciación entre las NRF24L01 con y sin PA+LNA con y sin antena

La versión con PA+LNA y antena externa ofrece mayor alcance y estabilidad que la versión estándar.

Versión	PA (Power Amplifier)	LNA (Low Noise Amplifier)	Antena	Alcance aproximado	Consumo
NRF24L01 estándar (sin PA/LNA, sin antena)	No	No	PCB integrada	10–30 m en interiores	Muy bajo
NRF24L01 con antena externa (sin PA/LNA)	No	No	Antena SMA	50–100 m	Bajo
NRF24L01 con PA+LNA y antena externa	Sí	Sí	Antena SMA	500–1000 m en condiciones óptimas	Más alto

PA (Power Amplifier): Incrementa la potencia de transmisión, extendiendo el alcance.

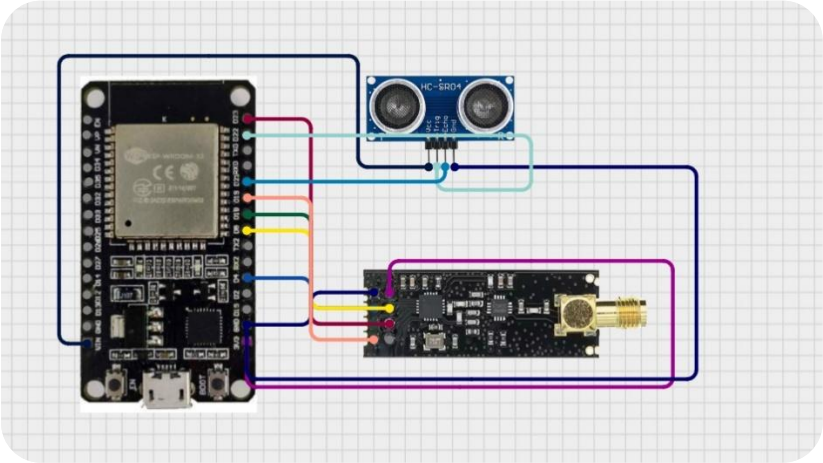
LNA (Low Noise Amplifier): Mejora la sensibilidad de recepción, reduciendo interferencias.

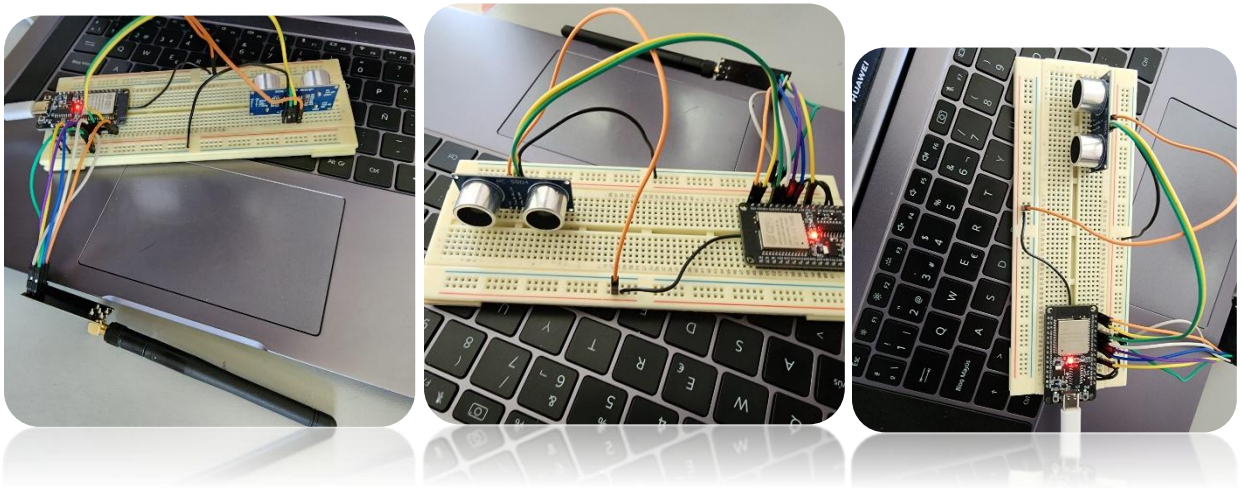
Antena externa: Proporciona mejor ganancia y estabilidad frente a obstáculos.

Circuitos

1. Transmisor

a) Diseño





Este circuito ha sido sencillo, donde los pines se conectaron como de costumbre a la antena y tras eso, el sensor HC-SR04 sin nada elaborado.

b) Código

Librerias que permiten la comunicación de los NRF24L01 y facilita su manejo

```
#include <SPI.h>

#include <RF24.h>
```

Aqui se define los pines del ESP-32 para controlar el módulo NRF24L01, el sensor ultrasónico e inicia una variable para guardar la distancia que lee el sensor

```
#define CE_PIN 4

#define CSN_PIN 5

#define echo 21

#define trigger 22

int distancia;
```

Se crea el objeto radio con los pines definidos y se crea una dirección de comunicación que se usara para ambos dispositivos

```
RF24 radio(CE_PIN, CSN_PIN);

const byte direccion[6] = "00001";
```

Este metodo hace funcionar el sensor ultrasónico, conforme la información llegue mediante los pines del echo y trigger, activa el pin del trigger para que emita el sonido ultrasónico y posteriormente active el pin del echo para escuchar el eco del sonido ultrasónico y devuelve la distancia captada

```
long readUltrasonicDistance(int triggerPin, int echoPin){

  pinMode(triggerPin, OUTPUT);

  digitalWrite(triggerPin, LOW);

  delayMicroseconds(2);

  digitalWrite(triggerPin, HIGH);
```

```
  delayMicroseconds(10);
  digitalWrite(triggerPin, LOW);
  pinMode(echoPin, INPUT);
  return pulseIn(echoPin, HIGH);
}
```

Se configurara el setup con una comunicación serial en 115200 y se inicia el pin para controlar el boton

```
void setup() {
Serial.begin(115200);
 pinMode(BOTON_PIN, INPUT_PULLUP);
```

Esta sentencia inicia el modulo NRF24L01

```
if (!radio.begin()) {
Serial.println("NRF24 NO detectado");
while (1);
}
```

Se configuran valores para la comunicación de los módulos. Se usara el canal 69 en 2.4 GHz, configurado para que use la máxima distancia posible y que use una velocidad de 250KBps para tener una transmisión mas confiable y estable.

```
radio.setChannel(69);
radio.setPALevel(RF24_PA_MAX);
radio.setDataRate(RF24_250KBPS);
```

Se abre el canal detransmision y se active el modulo para que funcione como transmisor en modo no escuchar.

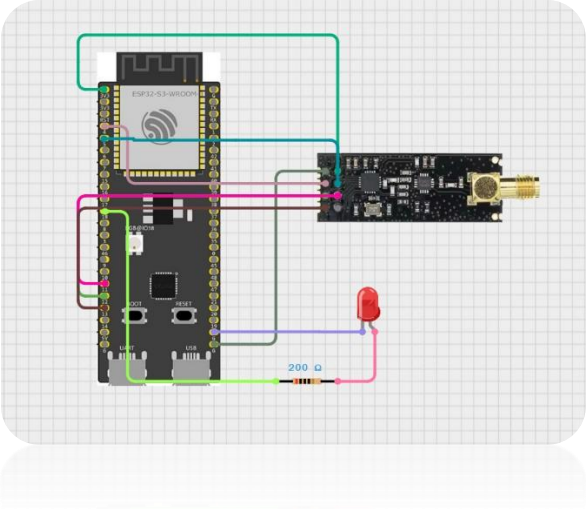
```
radio.openWritingPipe(direccion);
radio.stopListening();
Serial.println("Transmisor listo (ESP32)");
}
```

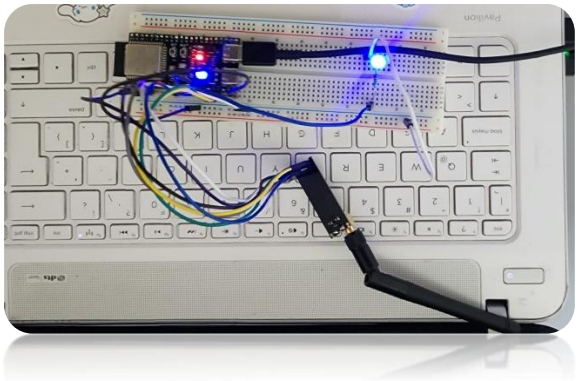
Se hace un calculo para que la distancia que devolvió el método del sensor ultrasónico se convierta en una distancia en cm. Despues se envia la variable con radio write y si se envia de forma correcta devuelve un mensaje de confirmación, si hay algún error devuelve mensaje de error

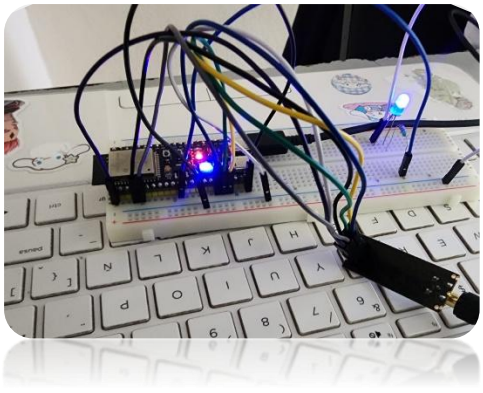
```
void loop() {
distancia = 0.01723 * readUltrasonicDistance(trigger, echo);
  if (radio.write(&distancia, sizeof(distancia))){
    Serial.print("Distancia enviada:");
  }
  else {
    Serial.println("Error");
  }}
```

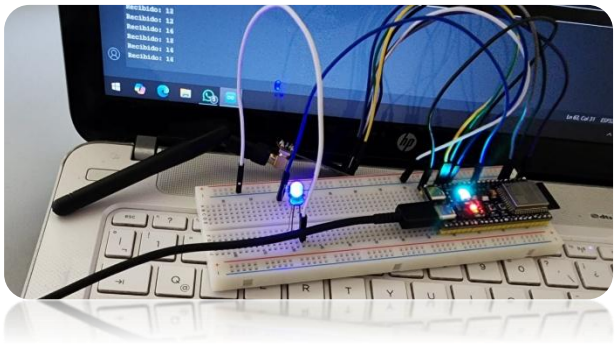
2. Receptor

c) Diseño









Este circuito se mantuvo idéntico al trabajo anterior, se enlazan una antena y un led con el ESP32. De esta forma, el LED integrado en el ESP32 funciona como corroborador de la conexión entre los ESP32 y el LED adicional, para representar la respuesta con el sensor.

d) Código

Librerias que permiten la comunicación de los NRF24L01 y facilita su manejo

```
#include <SPI.h>

#include <RF24.h>
```

Aqui se define los pines del ESP-32 para controlar el módulo NRF24L01 y define un pin para controlar el LED

```
#define CE_PIN 4

#define CSN_PIN 5

#define SCK_PIN 12

#define MISO_PIN 13

#define MOSI_PIN 11

#define pinLed 18
```

Variables para almacenar la distancia recibida y definir la intensidad del LED

```
int brillo =0;

int distancia = 0;
```

Se crea el objeto radio con los pines definidos y se crea una dirección de comunicación que se usara para ambos dispositivos

```
RF24 radio(CE_PIN, CSN_PIN);

const byte direccion[6] = "00001";
```

Se configurara el setup con una comunicación serial en 115200, booleano que controla el encendido del led

```
void setup() {
```

```
Serial.begin(115200);
bool encendido = false;
```

Se inicia la comunicación SPI con los pines que se definieron

```
SPI.begin(SCK_PIN, MISO_PIN, MOSI_PIN);
```

Esta sentencia inicia el modulo NRF24L01,

```
if (!radio.begin()) {
Serial.println("NRF24 NO detectado");
```

Se utiliza el led integrado para representar los distintos estados de la comunicación, en este caso rojo si ocurre algún error de hardware

```
#ifdef RGB_BUILTIN
// Rojo si hay error de hardware
rgbLedWrite(RGB_BUILTIN, RGB_BRIGHTNESS, 0, 0);
#endif
while (1);
}
Serial.println("NRF24 detectado");
```

Se configuran valores para la comunicación de los módulos. Se usara el canal 69 en 2.4 GHz, configurado para que use la máxima distancia posible y que use una velocidad de 250KBps para tener una transmisión más confiable y estable.

```
radio.setChannel(69);
radio.setPALevel(RF24_PA_MAX);
radio.setDataRate(RF24_250KBPS);
```

Abre el canal de comunicación con una dirección definida y activa el modo de escucha

```
 radio.openReadingPipe(0, direccion);
 radio.startListening();
```

Verifica si hay datos disponibles para leer, si hay los muestra en el puerto serial

```
Void loop() {
 if (radio.available()) {
radio.read(&distancia, sizeof(distancia));

 Serial.print("Recibido: ");
 Serial.println(texto);
```

Se cambia el estado del led para indicar si esta recibiendo señal o si hay algún error

```
encendido = !encendido;
 }
 #ifdef RGB_BUILTIN
 if (encendido) {
 rgbLedWrite(RGB_BUILTIN, 0, RGB_BRIGHTNESS, 0);
} else {
 rgbLedWrite(RGB_BUILTIN, 0, 0, 20);
 }
```

Se hace un map para controlar la intensidad del LED conforme la distancia que recibe mediante la transmisión de datos, siendo 255 cuando mas cercana la distancia y apagándose cuando esta fuera del rango de 50 cm.

```
brillo = map(distancia, 0, 50, 255, 0);
  brillo = constrain(brillo, 0, 255);
  if (distancia > 0) {
    analogWrite (pinLed, brillo);
  }
  delay(10);
 #endif
}
```

Conclusiones

Ramírez Rodríguez Martín de Jesús:

La incorporación del sensor de proximidad al esquema de comunicación entre dos ESP32 demostró la versatilidad y escalabilidad de este tipo de proyectos, como en las automatizaciones. A partir de una interacción sencilla se hizo evidente el potencial de integrar múltiples dispositivos y sensores para construir sistemas más robustos, útiles tanto en entornos educativos como en aplicaciones prácticas de automatización.

Santacruz Rodarte Cristian Alexis:

En conclusión, con la actividad presentada aprendimos los cimientos básicos en la creación de redes con antenas NRF24, la complementación con el equipo que recibía la señal ayudo amostrar ambos lados en la creación de redes Bluetooth, además de la creación de aplicaciones con el fin de controlar medios por la misma.

Pedroza Pérez Emmanuel:

Se implemento un dispositivo adicional al circuito el cual puede resolver varios problemas reales, aprendimos que el funcionamiento de las señales inalámbricas tienen muchas aplicaciones en la vida cotidiana y son aplicables a la resolución de problemas cotidianos

Enlace a GitHub

Este enlace que se muestra a continuación, muestra tanto el repositorio donde se encuentra la documentación misma, los códigos empleados para esta práctica y su video de evidencia:

https://github.com/MartinITAAgs/Tecnolog-as-Inal-mbricas-12pm/tree/main/3_Pr%C3%A1ctica%20NFR24L01%20-%20Env%C3%ADo%20de%20datos%20con%20generador%20de%20se%C3%B1al